# Argus: The Deterministic Security Layer for Enterprise AI Compliance

**Author:** Yan Tandeta

**Date:** May 2026

## Executive Summary

As enterprises rapidly adopt agentic AI, a structural security problem has emerged: traditional security primitives (firewalls, IAM, egress controls) are deterministic, while Large Language Models (LLMs) are inherently probabilistic. Relying on system prompts, guardrails, and "please don't" instructions baked into the context window is insufficient for production workloads and fails to meet rigorous audit standards.

**Argus** solves this by providing a deterministic security layer that wraps agent execution. It ensures that every tool call—whether a database query, API request, file read, or email send—passes through a gauntlet of deterministic Python code that the LLM never sees and cannot influence. By operating the LLM *inside* Argus rather than beside it, organizations can achieve up to 90% technical compliance with major frameworks like SOC 2, ISO 27001, HIPAA, and PCI-DSS out of the box.

This document provides a comprehensive analysis of the Argus framework, focusing on its architecture, security gates, and direct mapping to enterprise compliance standards.

---

## The Structural Security Problem in Agentic AI

In traditional software architecture, security controls are deterministic. A database role either allows a query or denies it. A firewall either permits a packet or drops it. These controls do the same thing every time, making them auditable, testable, and provable.

Agentic AI introduces a paradigm shift. The LLM decides at runtime which tool to call, with what arguments, and against which data. The industry's initial response to securing these systems has been to use more probabilistic mechanisms: system prompts, guardrails, and context-window instructions.

However, this approach is fundamentally flawed:

1. **Prompt Injection Vulnerabilities:** As demonstrated by numerous prompt-injection attacks, models can be manipulated into ignoring their instructions.
2. **Audit Failures:** Probabilistic guardrails do not pass rigorous compliance audits because their behavior cannot be mathematically guaranteed.

3. **Lack of Enforcement:** A model that is told not to exfiltrate data can still do so if it is tricked into calling an egress tool with malicious arguments.

---

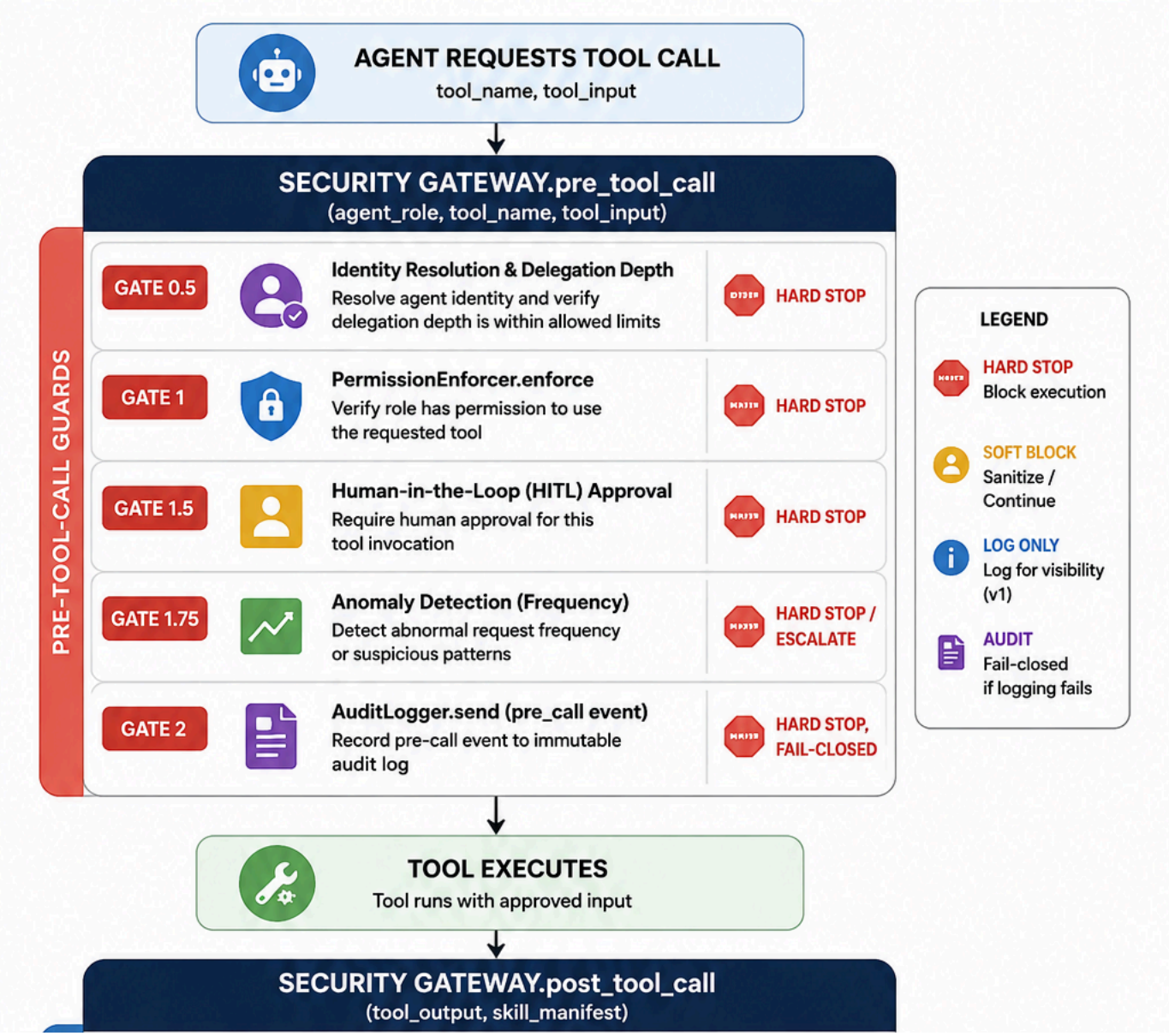
## The Argus Principle: Deterministic Enforcement

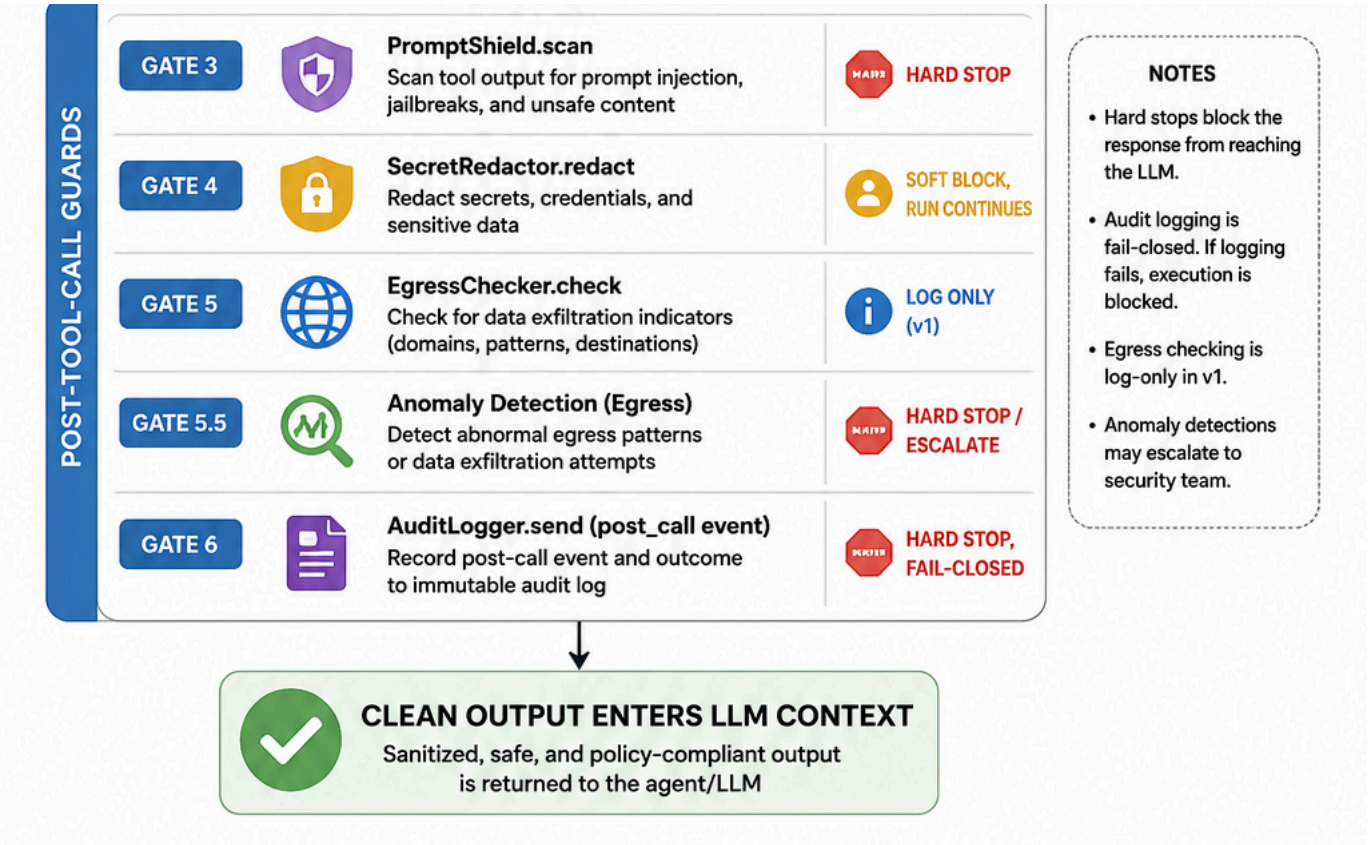
The core philosophy of Argus is simple but profound: **The LLM operates inside Argus, not beside it.**

Argus enforces security through a deterministic execution path. No LLM output, hallucination, or adversarial prompt can bypass the security gates because these components are not part of the LLM's execution. They are executed before and after every tool call in deterministic Python code.

### The Execution Path

The execution path through a tool call in Argus is strictly ordered and fail-closed:






The LLM receives the output only after it has passed the post-call gates. It cannot influence the permission rules, the prompt shield patterns, or the audit log.

---

## The Six Security Gates (Fail-Closed by Default)

Argus enforces a series of ordered gates on every single tool call. Every gate is fail-closed; a violation blocks the tool call rather than merely warning and continuing.

### 1. Identity and Delegation Control (Gate 0.5)

Every call carries a `caller_id` and `hop_depth` propagated through Python `ContextVars` across multi-agent delegation chains.

- **Structural Isolation:** A worker agent scoped to read-only access cannot call an admin tool, even if its supervisor is an admin.
- **Privilege Escalation Prevention:** Privilege escalation across agents is structurally impossible. The system enforces a maximum delegation depth (default is 3) to prevent runaway agent chains.

### 2. Permission Enforcement (Gate 1)

Argus uses a Casbin-based Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) model.

- **Declarative Policy:** Permissions are declared in a single YAML file (e.g., `argus.yaml`). There is no Python code required for policy definition, no embedded policy strings, and no LLM-mediated decisions.
- **Default Deny:** The policy effect requires at least one matching allow rule. A role with no rules denies all tool calls.

## 3. Human-in-the-Loop (HITL) (Gate 1.5)

For sensitive operations, tools can be marked with `require_approval: true`.

- **Terminal Approval:** A human operator must type `approve` or `deny` at the terminal.
- **Fail-Closed:** The default action on timeout or invalid input is `deny`. Both paths are strictly audit-logged.

## 4. Anomaly Detection (Gates 1.75 & 5.5)

Argus implements a per-agent Exponentially Weighted Moving Average (EWMA) and z-score statistical anomaly detection engine.

- **Real-Time Catch:** An agent that normally makes 50 calls an hour and suddenly makes 10,000 gets caught in real-time, before cost alarms fire.
- **Deterministic Math:** The detection uses standard library statistics (`statistics.stdev`)—no complex machine learning models, no `scipy`, and no probabilistic inference. It operates entirely in-process.

## 5. Tamper-Evident Audit Logging (Gates 2 & 6)

Every event is written to a hash-chained JSONL file managed by an out-of-process daemon.

- **Cryptographic Integrity:** Each entry is canonicalized and hashed with SHA-256, chaining to the previous hash.
- **Isolation:** A compromised agent cannot silence its own audit trail because the log process communicates over a Unix domain socket that the agent process does not own. If the socket is unavailable, the system fails closed (`AuditUnavailableError`).

## 6. Prompt Injection, Secret Redaction, and Egress (Gates 3, 4, & 5)

Argus enforces controls aligned with the OWASP LLM Top 10:

- **Prompt Shield:** Scans tool outputs against 14 OWASP-aligned patterns (e.g., "ignore previous instructions") compiled at startup. It strips zero-width Unicode obfuscation characters before scanning.
- **Secret Redaction:** Applies regex and entropy scanning to redact API keys, tokens, and PII (like emails) from tool outputs before they reach the LLM context. This is a soft block;

the run continues with sanitized text.

- **Egress Control:** Compares hostnames against a declared allowlist to prevent data exfiltration.

---

## Framework and Model Agnosticism

Argus is designed as a universal security control plane. It does not force organizations to abandon their existing agent frameworks or model providers.

- **Python Agents:** Argus provides drop-in adapters for LangChain, CrewAI, AutoGen, and any Model Context Protocol (MCP) server.
- **Non-Python Agents:** The `argus serve` command exposes a REST `/tool-call` endpoint. Any language that speaks JSON (Go, TypeScript, Rust, Java) gets the full security stack.
- **Any LLM:** Argus supports Anthropic, OpenAI, Llama, or anything LiteLLM supports, including local Ollama models.

By adopting Argus, enterprises gain a single, unified security control plane across every framework they are already using.

---

## Enterprise Compliance Mapping

Argus is designed to map directly to major enterprise compliance frameworks. While no software library can provide 100% compliance (as policies, training, and incident response runbooks are inherently human/operational), Argus delivers the technical controls necessary to satisfy auditors for the remaining 90%.

The following table maps Argus's deterministic controls to specific compliance standards:

| Compliance Standard | Relevant Controls | How Argus Delivers |
|---|---|---|
| **SOC 2 (CC6, CC7, CC8)** | Logical Access, System Monitoring, Change Management | Per-agent Casbin RBAC (Gate 1); Tamper-evident hash-chained audit log (Gates 2/6); Change-controlled skills with SHA-256 verification. |
| **ISO 27001 (Annex A.5, A.8, A.12)** | Information Security Policies, Asset Management, Operations Security | Declarative YAML access control; Out-of-process operational logging; EWMA + z-score anomaly monitoring (Gates 1.75/5.5). |

| | | |
|---|---|---|
| **HIPAA ( § 164.312)** | Technical Safeguards: Access Control, Audit Controls, Integrity | Strict identity propagation ( `caller_id` ); Cryptographic audit controls; Secret/PII redaction (Gate 4) preventing PHI leakage into LLM context. |
| **PCI-DSS (Req 7, 8, 10, 11)** | Least Privilege, Identification, Logging, Testing | Default-deny RBAC (Req 7); Multi-agent identity tracking (Req 8); Comprehensive JSONL logging (Req 10); Prompt injection intrusion detection (Req 11). |
| **NIST AI RMF (Govern, Manage)** | Risk Management, Continuous Monitoring | Fail-closed risk treatment across all gates; Continuous statistical monitoring of tool frequency and egress volume. |
| **EU AI Act (High-Risk Art. 9-15)** | Risk Management, Logging, Human Oversight | Deterministic risk mitigation; Tamper-evident logging; Terminal-based Human-in-the-Loop (HITL) approval for sensitive tools (Gate 1.5). |

### The "90%" Claim Explained

When an auditor asks, "Show me how you control what the agent can do," a probabilistic system prompt is insufficient. Argus provides the running code that proves control.

Of the controls that are technically about how the AI agent system operates—access, logging, change management, monitoring, incident detection—Argus addresses approximately 90%. The remaining 10% consists of operational evidence: employee training, vendor management, and written policies.

As the Argus philosophy states: **"Argus runs the system. You run the program."**

---

## Conclusion

Deploying agentic AI in enterprise environments requires moving beyond probabilistic guardrails. Argus provides the deterministic enforcement layer necessary to secure AI workloads and pass rigorous compliance audits. By wrapping agent execution in a fail-closed, mathematically verifiable security gateway, Argus ensures that AI agents operate safely, securely, and within the bounds of enterprise policy.

For organizations where security reviews are taking longer than engineering, Argus offers a drop-in, framework-agnostic solution to bridge the gap between AI innovation and enterprise compliance.

---

*Argus is open source, Apache 2.0, Python 3.12+. v0.4 ships with multi-agent identity and anomaly detection.*